

UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS

FACULTAD DE INGENIERÍA TAMPICO

NOMBRE DEL PROYECTO:

**DESARROLLO DE UN ANALIZADOR LÉXICO PARA UN LENGUAJE
PERSONALIZADO**

NOMBRE DEL LENGUAJE:

STATSCRIPT

DOCENTE:

MTRO. MUÑOZ QUINTERO DANTE ADOLFO

ASIGNATURA:

PROGRAMACIÓN DE SISTEMAS DE BASE I

GRADO Y GRUPO:

8 ° “K”

INTEGRANTES:

NOMBRE	MATRÍCULA
CABRALES CRUZ RACIEL JULIAN	2223330141
GUZMÁN SÁNCHEZ ALEX ARATH	2223330162
MUÑOZ PERALES LUIS GONZALO	2193283019
SAENZ RICO PEREZ MARCO FERNANDO	2223330190

REPOSITORIO:

<https://github.com/Sync-Thread/analizador-lexico-statscript>

Índice

1. Introducción	3
2. Objetivos	4
2.1. Objetivo General.....	4
2.2. Objetivos Específicos.....	4
3. Especificación del Lenguaje	5
3.1. Nombre y justificación.....	5
3.2. Ejemplos ilustrativos de código.....	5
Ejemplo 1 — Análisis de ventas:	5
Ejemplo 2 — Condicional con operadores:	5
Ejemplo 3 — Expresión aritmética simple:	5
3.3. Catálogo de tokens.....	6
4. Implementación.....	8
4.1. Definición de tokens y palabras reservadas — <code>tokens.py</code>	8
4.2. Reglas léxicas — <code>rules.py</code>	9
4.3. Lógica del analizador léxico — <code>scanner.py</code>	10
4.4. Modelo de datos y clasificación de errores — <code>token_model.py</code>	13
4.5. Punto de entrada y presentación de resultados — <code>main.py</code>	14
4.6. Nodos del árbol de sintaxis abstracta — <code>ast_nodes.py</code>	17
4.7. Analizador sintáctico — <code>parser.py</code>	19
5. Demostración Funcional.....	22
5.1. Ejecución con programa válido	22
5.2. Ejecución con errores léxicos	27
5.3. Caso con identificadores inválidos y símbolos desconocidos.....	29
5.4. Segundo programa válido.....	32
5.5. Manejo de archivo no encontrado	34
5.6. Modo consola	35
6. Conclusiones	37
Trabajo futuro	38
7. Referencias.....	39

1. Introducción

El análisis léxico constituye la primera fase dentro de la construcción de un compilador o intérprete. Su función consiste en recibir una secuencia de caracteres y transformarla en una secuencia de tokens, es decir, las unidades mínimas con significado del lenguaje. Comprender esta etapa de forma práctica resulta fundamental en la formación de todo ingeniero en sistemas computacionales que aspire a construir herramientas de procesamiento de lenguajes.

En ese contexto, el presente proyecto diseña e implementa un analizador léxico para **StatScript**, un lenguaje de dominio específico orientado al análisis estadístico de datos tabulares. Su nombre, derivado de Statistics y Script, refleja directamente su propósito principal, que es permitir que usuarios sin experiencia avanzada en programación expresen operaciones estadísticas de manera declarativa y en español. Entre sus capacidades se encuentran la carga y exportación de archivos CSV, los cálculos estadísticos descriptivos, el filtrado condicional, el ordenamiento, la agrupación y la generación de gráficas.

El lexer fue implementado en Python 3.12 sin dependencias externas, apoyándose únicamente en la biblioteca estándar de expresiones regulares (`re`) y el módulo `dataclasses`. Su diseño le permite transformar archivos de texto fuente con extensión `.sts` en una lista numerada de tokens, e informar con precisión la línea y columna de cualquier error léxico detectado.

En cuanto a la organización del documento, la Sección 2 presenta los objetivos del proyecto, mientras que la Sección 3 describe la especificación del lenguaje StatScript. Posteriormente, la Sección 4 contiene el código fuente comentado del lexer y la Sección 5 muestra capturas de ejecución. Finalmente, las Secciones 6 y 7 recogen las conclusiones del equipo y las referencias bibliográficas consultadas.

2. Objetivos

2.1. Objetivo General

Diseñar un lenguaje personalizado denominado StatScript e implementar un analizador léxico funcional en Python capaz de transformar programas fuente escritos en dicho lenguaje en listas de tokens, incorporando además un manejo preciso de errores léxicos.

2.2. Objetivos Específicos

1. Definir la especificación informal del lenguaje StatScript a través de ejemplos ilustrativos de código que cubran sus construcciones principales, entre las que se incluyen comandos, funciones estadísticas, operadores y literales.
2. Establecer un catálogo completo de tokens que contemple 33 palabras reservadas organizadas en 7 categorías semánticas, junto con identificadores, números enteros y flotantes, operadores aritméticos y relacionales, delimitadores y cadenas de texto.
3. Implementar el analizador léxico como un autómata manual basado en caracteres, asegurando tanto el reconocimiento correcto de todos los tokens definidos como la clasificación adecuada de palabras reservadas frente a identificadores de usuario.
4. Incorporar la detección y reporte de errores léxicos de tal forma que se indique el número de línea, la columna y una descripción clara del problema para cada error encontrado, sin detener el análisis ante el primer error.
5. Probar el lexer con al menos dos programas válidos y dos programas con errores léxicos que cubran los casos descritos en la especificación, con el objetivo de alcanzar una cobertura del 100 % en los tipos de token y los tipos de error.

3. Especificación del Lenguaje

3.1. Nombre y justificación

StatScript (Statistics + Script) es un lenguaje imperativo de propósito educativo diseñado para expresar operaciones de análisis estadístico sobre datos tabulares. Su sintaxis es declarativa y está escrita en español, lo que facilita la lectura y comprensión del código por parte de usuarios no especializados. El lenguaje ofrece soporte para la carga de archivos CSV, cálculos estadísticos descriptivos como promedio, mediana, moda, desviación, varianza y correlación, así como filtrado condicional, ordenamiento, agrupación y generación de distintos tipos de gráficas. Los archivos fuente de StatScript utilizan la extensión `.sts`.

3.2. Ejemplos ilustrativos de código

Ejemplo 1 — Análisis de ventas:

```
cargar "ventas_2024.csv";

calcular promedio de precio;
calcular mediana de precio;
calcular desviacion de precio;

filtrar precio > 100 y categoria == "electrónica";
ordenar precio descendente;
agrupar por departamento;

graficar barras de ventas;
exportar "reporte_final.csv";

mostrar datos;
```

Ejemplo 2 — Condicional con operadores:

```
filtrar calificacion >= 70;
filtrar asistencia > 80 o calificacion >= 90;
filtrar calificacion != 0;
```

Ejemplo 3 — Expresión aritmética simple:

```
resultado = (calificacion + extra) * 0.5;
calcular suma de calificacion / 100;
```

3.3. Catálogo de tokens

CATEGORÍA	TOKEN(S)	PATRÓN / EJEMPLOS	DESCRIPCIÓN
COMANDOS	CARGAR, MOSTRAR, CALCULAR, FILTRAR, ORDENAR, AGRUPAR, GRAFICAR, EXPORTAR	Palabras reservadas (case-insensitive)	Instrucciones principales del lenguaje
FUNCIONES ESTADÍSTICAS	PROMEDIO, MEDIANA, MODA, DESVIACION, VARIANZA, CONTEO, SUMA, MINIMO, MAXIMO, CORRELACION	Palabras reservadas	Cálculos estadísticos
TIPOS DE GRÁFICA	BARRAS, LINEAS, PASTEL, DISPERSION, HISTOGRAMA	Palabras reservadas	Tipos de visualización
PREPOSICIONES	DE, POR	Palabras reservadas	Conectores sintácticos
MODIFICADORES	ASCENDENTE, DESCENDENTE	Palabras reservadas	Orden de clasificación
OPERADORES LÓGICOS	Y, O, NO	Palabras reservadas	Operadores booleanos
OBJETOS DEL LENGUAJE	DATOS, COLUMNAS, REGISTROS	Palabras reservadas	Elementos del lenguaje
OP. ARITMÉTICOS	MAS, MENOS, MULTIPLICAR, DIVIDIR, MÓDULO	+, -, *, /, %	Operaciones matemáticas
OP. RELACIONALES	MAYOR, MENOR, MAYOR_IGUAL, MENOR_IGUAL, IGUAL, DIFERENTE	>, <, >=, <=, ==, !=	Comparaciones
ASIGNACIÓN	ASIGNAR	= (simple, sin ==)	Asignación de valores
DELIMITADORES	PUNTO_COMA, COMA, LPAREN, RPAREN	;, ,, (,)	Separadores y agrupadores
ENTERO	ENTERO	0, 42, 1024	Secuencia de dígitos sin punto decimal
FLOTANTE	FLOTANTE	3.14, 0.5, 2.0	Dígitos, punto, dígitos obligatorios
CADENA	CADENA	"hola", "valor"	Cadena entre comillas dobles
IDENTIFICADOR	ID	x, total, my_var	Letras o _ seguidas de letras, dígitos o _
COMENTARIO DE LÍNEA	(se descarta)	-- esto es un comentario	Desde -- hasta fin de línea
COMENTARIO DE BLOQUE	(se descarta)	<-- ... -->	Multilínea, desde <- - hasta -->

Identificadores válidos: comienzan con letra (a-z, A-Z) o guión bajo (_), seguidos de cero o más letras, dígitos o guiones bajos.

Identificadores inválidos (generan error): comienzan con dígito (123abc), contienen caracteres especiales no permitidos (my-var, x@y).

Total: 33 palabras reservadas organizadas en 7 categorías semánticas.

4. Implementación

A continuación, se presenta el código fuente de los módulos que conforman el analizador léxico. Los fragmentos han sido seleccionados para ilustrar los aspectos de mayor relevancia conceptual, mientras que el código completo puede consultarse en el repositorio GitHub indicado en la portada.

4.1. Definición de tokens y palabras reservadas — `tokens.py`

Este módulo define las categorías de tipos de token mediante conjuntos inmutables (`frozenset`) y un diccionario centralizado llamado `KEYWORD_LEXEMES`, el cual permite al scanner reclasificar identificadores como palabras reservadas. La decisión de usar constantes centralizadas garantiza que tanto el lexer como futuras fases del compilador compartan exactamente los mismos identificadores, evitando inconsistencias.

`src/tokens.py` — tipos de token y palabras reservadas

```
# Comandos principales del lenguaje
COMMANDS = frozenset({
    'CARGAR', 'MOSTRAR', 'CALCULAR', 'FILTRAR',
    'ORDENAR', 'AGRUPAR', 'GRAFICAR', 'EXPORTAR',
})

# Funciones estadísticas
STAT_FUNCTIONS = frozenset({
    'PROMEDIO', 'MEDIANA', 'MODA', 'DESVIACION', 'VARIANZA',
    'CONTEO', 'SUMA', 'MINIMO', 'MAXIMO', 'CORRELACION',
})

# Tipos de gráfica
CHART_TYPES = frozenset({
    'BARRAS', 'LINEAS', 'PASTEL', 'DISPERSION', 'HISTOGRAMA',
})

# Preposiciones, modificadores, operadores lógicos, objetos
PREPOSITIONS = frozenset({'DE', 'POR'})
MODIFIERS = frozenset({'ASCENDENTE', 'DESCENDENTE'})
LOGICAL_OPERATORS = frozenset({'Y', 'O', 'NO'})
LANGUAGE_OBJECTS = frozenset({'DATOS', 'COLUMNAS', 'REGISTROS'})

# Operadores y asignación
RELATIONAL_OPERATORS = frozenset({
```

```
'MAYOR', 'MENOR', 'MAYOR_IGUAL', 'MENOR_IGUAL', 'IGUAL', 'DIFERENTE',
})
ARITHMETIC_OPERATORS = frozenset({
    'MAS', 'MENOS', 'MULTIPLICAR', 'DIVIDIR', 'MODULO',
})
ASSIGNMENTS = frozenset({'ASIGNAR'})

# Literales, identificadores, separadores, control
LITERALS = frozenset({'ENTERO', 'FLOTANTE', 'CADENA'})
IDENTIFIERS = frozenset({'ID'})
SEPARATORS = frozenset({'PUNTO_COMA', 'COMA', 'LPAREN', 'RPAREN'})
OTHERS = frozenset({'NEWLINE', 'ERROR', 'EOF'})

# Mapeo centralizado: lexema en minúsculas -> tipo de token.
# El Scanner usa este diccionario para reconocer palabras reservadas.
KEYWORD_LEXEMES = {
    'cargar': 'CARGAR', 'mostrar': 'MOSTRAR', 'calcular': 'CALCULAR',
    'filtrar': 'FILTRAR', 'ordenar': 'ORDENAR', 'agrupar': 'AGRUPAR',
    'graficar': 'GRAFICAR', 'exportar': 'EXPORTAR',
    'promedio': 'PROMEDIO', 'mediana': 'MEDIANA', 'moda': 'MODA',
    # ... (33 entradas en total)
}

TOKEN_TYPES = { ... } # Diccionario de categorías
ALL_TOKEN_TYPES = frozenset().union(*TOKEN_TYPES.values())
```

4.2. Reglas léxicas — *rules.py*

Este archivo centraliza todas las expresiones regulares del lenguaje. El orden en que se definen las reglas resulta determinante para el correcto funcionamiento del lexer. Por ejemplo, la regla de **FLOTANTE** debe evaluarse antes que la de **ENTERO** para evitar que una entrada como **3.14** se tokenice erróneamente como tres tokens separados. De manera análoga, los operadores de dos caracteres (**>=**, **<=**, **==**, **!=**) se evalúan antes que los de un solo carácter.

grammar/statscript_rules.py — expresiones regulares ordenadas

```
import re

# El orden importa: primero los patrones más específicos.
# Prioridad:
# 1. Espacios y comentarios (se descartan, no generan token)
# 2. Cadenas de texto
# 3. Flotantes antes que enteros
# 4. Identificadores / palabras clave
```

```
# 5. Operadores de dos caracteres (>=, <=, ==, !=)
# 6. Operadores de un carácter
# 7. Delimitadores
# 8. Cualquier otra cosa → ERROR

RULES = {
    "WHITESPACE": re.compile(r'[\t\r]+'),
    "LINE_COMMENT": re.compile(r'--[^\n]*'),
    "BLOCK_COMMENT": re.compile(r'<--.*?-->', re.DOTALL),

    "FLOTANTE": re.compile(r'\d+\.\d+'),
    "ENTERO": re.compile(r'\d+'),
    "CADENA": re.compile(r'"[^\"]*"'),

    "IDENTIFIER": re.compile(r'[a-zA-Z_][a-zA-Z0-9_]*'),

    "MAYOR_IGUAL": re.compile(r'>='),
    "MENOR_IGUAL": re.compile(r'<='),
    "IGUAL": re.compile(r'=='),
    "DIFERENTE": re.compile(r'!='),

    "MAS": re.compile(r'\++'),
    "MENOS": re.compile(r'\-+'),
    # ... operadores simples y delimitadores
}

# Patrón para detectar identificadores mal formados:
# secuencias que empiezan con dígitos y luego tienen letras
INVALID_IDENTIFIER = re.compile(r'\d+[a-zA-Z_][a-zA-Z0-9_]*')
```

4.3. Lógica del analizador léxico — `scanner.py`

Este módulo implementa el bucle principal del lexer. En cada iteración, el scanner intenta hacer coincidir las reglas en orden desde la posición actual del texto fuente. Cuando ninguna regla coincide, se emite un token de tipo **ERROR** con la ubicación exacta del carácter problemático y se avanza una posición para evitar caer en un bucle infinito. A lo largo de todo el proceso, el scanner mantiene un registro actualizado del número de línea y columna, lo que permite generar mensajes de error con ubicación precisa.

`src/scanner.py` — analizador léxico principal

```

from typing import List, Any
from token_model import Token
from tokens import KEYWORD_LEXEMES

class Scanner:
    def __init__(self, source: str):
        self.source = source
        self.tokens: List[Token] = []
        self.begin = 0    # inicio del token actual
        self.current = 0   # posición de lectura
        self.line = 1
        self.column = 1
        self.keywords = KEYWORD_LEXEMES

    def scan_tokens(self) -> List[Token]:
        while not self.is_at_end():
            self.begin = self.current
            self.scan_token()
            self.add_token("EOF", "")
        return self.tokens

    def scan_token(self):
        c = self.advance()

        if c == ';': self.add_token("PUNTO_COMA")
        elif c == ',': self.add_token("COMA")
        elif c == '(': self.add_token("LPAREN")
        elif c == ')': self.add_token("RPAREN")

        elif c == '=':
            self.add_token("IGUAL" if self.match('=') else "ASIGNAR")
        elif c == '!':
            if self.match('='): self.add_token("DIFERENTE")
            else: self.add_error(c, "Símbolo no reconocido")
        elif c == '>':
            self.add_token("MAYOR_IGUAL" if self.match('>') else "MAYOR")
        elif c == '<':
            if self.match('<'): self.add_token("MENOR_IGUAL")
            elif self.peek() == '-' and self.peek_next() == '-':
                self.advance(); self.advance()
                self.block_comment()
            else: self.add_token("MENOR")

        # Operadores aritméticos
        elif c == '+': self.add_token("MAS")
        elif c == '-':
            if self.match('-'): self.line_comment()
            else: self.add_token("MENOS")
        elif c == '*': self.add_token("MULTIPLICAR")
        elif c == '/': self.add_token("DIVIDIR")

```

```
elif c == '%': self.add_token("MODULO")

elif c in '\t\r': return
elif c == '\n':
    self.add_token("NEWLINE")
    self.line += 1; self.column = 1

elif c == '": self.string_literal()
elif c.isdigit(): self.number_literal()
elif self.is_alpha(c): self.identifier()

else:
    self.add_error(c, "Símbolo no reconocido")

# --- Métodos auxiliares de navegación ---

def advance(self) -> str:
    # Consume y retorna el carácter actual, avanzando el puntero
    ...

def match(self, expected: str) -> bool:
    # Consume el siguiente carácter solo si coincide con expected
    ...

def peek(self) -> str:
    # Retorna el carácter actual sin consumirlo (lookahead 1)
    ...

def peek_next(self) -> str:
    # Retorna el carácter siguiente al actual (lookahead 2)
    ...

# --- Métodos de reconocimiento de tokens ---

def string_literal(self):
    # Consume caracteres hasta encontrar '"' o '\n'.
    # Si encuentra '"', emite token CADENA.
    # Si llega a '\n' o EOF, emite error "Cadena sin cerrar".
    ...

def number_literal(self):
    # Consume dígitos. Si encuentra '.' seguido de dígitos → FLOTANTE.
    # Si después de los dígitos hay una letra → error "Identificador inválido".
    # En otro caso → ENTERO.
    ...

def identifier(self):
    # Consume letras, dígitos y '_'.
    # Busca el lexema en KEYWORD_LEXEMES:
    # Si coincide → emite el tipo de palabra reservada.
    # Si no → emite ID.
```

```
...

def block_comment(self):
    # Consume caracteres hasta encontrar '-->'.
    # Actualiza línea/columna en saltos de línea internos.
    # Si llega a EOF sin cierre → error "Comentario de bloque sin cerrar".
    ...
```

4.4. Modelo de datos y clasificación de errores — *token_model.py*

Antes de presentar el punto de entrada del programa, conviene describir el modelo de datos que sustenta todo el sistema. La clase `Token` fue diseñada como un `dataclass` de Python y representa la unidad léxica fundamental que el analizador produce. Cada instancia almacena el tipo de token, su valor original, la línea y columna donde fue encontrado, y opcionalmente un mensaje de error cuando se trata de un token inválido. Esta separación resulta relevante porque permite que el Scanner delegue al modelo la responsabilidad de categorizar y filtrar tokens, manteniendo así una arquitectura modular. Además, el campo `error_message` es el que permite al sistema diferenciar con precisión los distintos tipos de error léxico que StatScript contempla.

src/token_model.py — modelo de datos del token

```
from dataclasses import dataclass
from typing import Any
from tokens import (ALL_TOKEN_TYPES, COMMANDS, STAT_FUNCTIONS, ...)

@dataclass
class Token:
    """Unidad léxica reconocida por el analizador."""
    type: str
    value: Any
    line: int
    column: int
    position: int
    error_message: str = None

    def __str__(self) -> str:
        return f"Token({self.type}, {repr(self.value)}, linea={self.line}, col={self.column})"

    # Métodos de consulta por categoría
    def is_error(self) -> bool: return self.type == "ERROR"
    def is_command(self) -> bool: return self.type in COMMANDS
    def is_stat_function(self) -> bool: return self.type in STAT_FUNCTIONS
    def is_literal(self) -> bool: return self.type in LITERALS
    def is_id(self) -> bool: return self.type in IDENTIFIERS
    # ... métodos adicionales para cada categoría de token
```

Gracias a esta estructura, el Scanner puede distinguir cuatro tipos de error léxico, cada uno con su mensaje descriptivo correspondiente.

1. **Símbolo no reconocido.** Se genera cuando el Scanner encuentra caracteres que no forman parte de la gramática de StatScript, como \$, @, #, ~ o &.
2. **Identificador inválido.** Ocurre ante secuencias que inician con un dígito seguido de letras, por ejemplo 123abc o 456columna, las cuales no constituyen identificadores válidos.
3. **Cadena sin cerrar.** Se reporta cuando una comilla doble de apertura no encuentra su comilla de cierre antes del fin de línea.
4. **Comentario de bloque sin cerrar.** Se emite cuando un delimitador de apertura <-- no encuentra su correspondiente --> antes del fin de archivo.

4.5. Punto de entrada y presentación de resultados — *main.py*

El módulo principal actúa como orquestador del flujo completo de ejecución. Su responsabilidad consiste en recibir la entrada del usuario (ya sea un archivo .sts o texto interactivo), invocar al Scanner para que procese el código fuente y, finalmente, formatear la salida de manera legible. La presentación se divide en dos bloques bien diferenciados, primero la lista numerada de tokens reconocidos y después los errores léxicos detectados junto con su ubicación exacta.

src/main.py — punto de entrada del programa

```
import sys
from scanner import Scanner

def print_tokens(tokens):
```

```
"""Imprime la lista de tokens numerada, excluyendo errores, NEWLINE y EOF."""
```

```
valid = [t for t in tokens if not t.is_error() and not t.is_newline() and not t.is_eof()]
```

```
if not valid:
```

```
    print(" (sin tokens)")
```

```
    return
```

```
for i, token in enumerate(valid, 1):
```

```
    print(f" {i}: ({token.type}, {token.value!r})")
```

```
def print_errors(tokens):
```

```
    """Imprime la lista de errores léxicos con línea, columna y descripción."""
```

```
    errors = [t for t in tokens if t.is_error()]
```

```
    if not errors:
```

```
        print(" Sin errores léxicos.")
```

```
        return
```

```
    for error in errors:
```

```
        msg = error.error_message or "Símbolo no reconocido"
```

```
        print(f" Línea {error.line}, Columna {error.column}: {msg} {error.value!r}")
```

```
def analyze(source, label=""):
```

```
    """Ejecuta el análisis léxico y muestra resultados."""
```

```
    print(f"\n{'=' * 60}")
```

```
    if label:
```

```
        print(f" Archivo: {label}")
```

```
    print(f" Resultado del análisis léxico")
```

```
    print(f"{'=' * 60}")
```

```
    scanner = Scanner(source)
```

```
    tokens = scanner.scan_tokens()
```

```
    print("\nToken List:")
```

```
    print_tokens(tokens)
```

```
print(f"\n{'-' * 60}")
print("Error List:")
print_errors(tokens)

def main():
    if len(sys.argv) > 1:
        filepath = sys.argv[1]
        try:
            with open(filepath, "r", encoding="utf-8") as f:
                source = f.read()
        except FileNotFoundError:
            print(f"Error: No se encontró el archivo '{filepath}'")
            sys.exit(1)
        analyze(source, label=filepath)
    else:
        print("StatScript — Analizador Léxico")
        print("Escribe tu programa StatScript (línea vacía para analizar):\n")
        lines = []
        while True:
            try:
                line = input()
            except EOFError:
                break
            if line == "":
                break
            lines.append(line)
        source = "\n".join(lines)
        analyze(source)

if __name__ == "__main__":
    main()
```

4.6. Nodos del árbol de sintaxis abstracta — `ast_nodes.py`

Una vez que el Scanner produce la lista de tokens, resulta necesario organizar esa información en una estructura que refleje la jerarquía del programa. Para ello se definió un conjunto de clases que representan los nodos del Árbol de Sintaxis Abstracta (AST). Cada tipo de sentencia de StatScript cuenta con su propia clase, lo que permite al parser construir una representación semántica del programa independiente de la sintaxis textual original.

El módulo utiliza `dataclasses` de Python para mantener las definiciones concisas y legibles. También incluye una función `imprimir_ast()` que recorre el árbol de forma recursiva y genera una salida indentada para facilitar la verificación visual del resultado del análisis sintáctico.

`src/ast_nodes.py` — definición de nodos del AST

```
from dataclasses import dataclass, field
from typing import List, Any

@dataclass
class NodoAST:
    """Nodo base del árbol de sintaxis abstracta."""
    pass

@dataclass
class Programa(NodoAST):
    """Representa un programa completo (lista de sentencias)."""
    sentencias: List[NodoAST] = field(default_factory=list)

# Sentencias del lenguaje
```

```
@dataclass
class CargarStmt(NodoAST):
    archivo: str = ""

@dataclass
class CalcularStmt(NodoAST):
    funcion: str = ""
    columna: str = ""

@dataclass
class FiltrarStmt(NodoAST):
    condicion: Any = None

@dataclass
class AsignacionStmt(NodoAST):
    nombre: str = ""
    expresion: Any = None

# Expresiones y condiciones
@dataclass
class Condicion(NodoAST):
    izquierda: Any = None
    operador: str = ""
    derecha: Any = None

@dataclass
class ExpresionBinaria(NodoAST):
    izquierda: Any = None
    operador: str = ""
    derecha: Any = None

@dataclass
class Literal(NodoAST):
    valor: Any = None
```

```
tipo: str = ""

@dataclass
class Identificador(NodoAST):
    nombre: str = ""

# ... clases adicionales para MostrarStmt, OrdenarStmt,
# AgruparStmt, GraficarStmt, ExportarStmt, CondicionCompuesta
```

4.7. Analizador sintáctico — `parser.py`

La **propuesta actual** del **parser** implementa la técnica de análisis recursivo descendente, en la que cada regla de la gramática EBNF descrita en la sección 3.4 se traduce directamente en un método del parser. El flujo de trabajo comienza filtrando los tokens irrelevantes para la sintaxis (`NEWLINE` y `ERROR`) y luego recorre la lista construyendo progresivamente el AST.

Cuando el parser encuentra una construcción que no coincide con ninguna regla gramatical, genera un error sintáctico y ejecuta un mecanismo de sincronización que avanza hasta el siguiente punto y coma para poder continuar con el análisis del resto del programa, de forma análoga a como el Scanner recupera el análisis léxico ante errores.

`src/parser.py` — analizador sintáctico recursivo descendente

```
from typing import List, Optional
from token_model import Token
from ast_nodes import *

class ErrorSintactico(Exception):
    """Representa un error encontrado durante el análisis sintáctico."""
    def __init__(self, mensaje, token=None):
        self.mensaje = mensaje
        self.token = token
```

```
self.linea = token.line if token else 0
self.columna = token.column if token else 0

class Parser:
    def __init__(self, tokens: List[Token]):
        self.tokens = [t for t in tokens if t.type not in ('NEWLINE', 'ERROR')]
        self.pos = 0
        self.errores: List[ErrorSintactico] = []

    def parse(self) -> Programa:
        """Punto de entrada. Retorna el AST completo."""
        programa = Programa()
        while not self._es_fin():
            try:
                stmt = self._sentencia()
                if stmt:
                    programa.sentencias.append(stmt)
            except ErrorSintactico as e:
                self.errores.append(e)
                self._sincronizar()
        return programa

    def _sentencia(self):
        """sentencia → comando PUNTO_COMA"""
        stmt = self._comando()
        self._esperar('PUNTO_COMA', "Se esperaba ';' al final de la sentencia")
        return stmt

    def _comando(self):
        """Despacha al método correspondiente según el token actual."""
        if self._verificar('CARGAR'): return self._cargar()
        elif self._verificar('CALCULAR'): return self._calcular()
        elif self._verificar('FILTRAR'): return self._filtrar()
        elif self._verificar('ID'): return self._asignacion()
```

```
# ... demás comandos: mostrar, ordenar, agrupar, graficar, exportar

def _condicion(self):
    """condicion → [NO] comparacion { (Y | O) [NO] comparacion }"""
    # Soporta el operador unario NO como prefijo
    ...

def _expresion(self):
    """expresion → termino { (MAS | MENOS) termino }"""
    ...

def _termino(self):
    """termino → factor { (MULTIPLICAR | DIVIDIR | MODULO) factor }"""
    ...

def _factor(self):
    """factor → ENTERO | FLOTANTE | CADENA | ID | LPAREN expresion RPAREN"""
    ...

def _sincronizar(self):
    """Avanza hasta el siguiente punto y coma para recuperarse del error."""
    while not self._es_fin():
        if self._actual().type == 'PUNTO_COMA':
            self._avanzar()
            return
        self._avanzar()
```

El punto de entrada del programa (*main.py*) fue actualizado para invocar al parser después del Scanner. Si el análisis léxico no detecta errores, se construye el AST y se muestra en la salida junto con los tokens. En caso de que existan errores léxicos, el análisis sintáctico se omite para evitar resultados inconsistentes.

5. Demostración Funcional

En esta sección se presentan las capturas obtenidas al ejecutar el lexer desde la terminal. Cada escenario fue probado mediante el comando ***python src/main.py*** desde el directorio raíz del proyecto.

5.1. Ejecución con programa válido

Para esta primera prueba se tokeniza el archivo `tests/valid/programa1.sts`, el cual implementa un análisis completo de ventas y no contiene errores léxicos de ningún tipo.

Archivo de entrada — `tests/valid/programa1.sts`:

```
-- Programa 1: Análisis completo de ventas
-- Este programa carga un archivo CSV, realiza cálculos estadísticos,
-- filtra y ordena los datos, y genera un reporte.

cargar "ventas_2024.csv";

-- Cálculos estadísticos básicos
calcular promedio de precio;
calcular mediana de precio;
calcular desviacion de precio;
calcular suma de ventas;
calcular conteo de registros;

-- Filtrado de datos
filtrar precio > 100 y categoría == "electrónica";
filtrar cantidad >= 5;

-- Ordenamiento
ordenar precio descendente;

-- Agrupación
```

```
agrupar por departamento;
```

```
-- Visualización
```

```
graficar barras de ventas;
```

```
graficar lineas de precio;
```

```
-- Exportar resultados
```

```
exportar "reporte_ventas.csv";
```

```
mostrar datos;
```

Ejecución y salida:

```
$ python src/main.py tests/valid/programa1.sts
```

```
=====
```

```
Archivo: tests/valid/programa1.sts
```

```
Resultado del análisis léxico
```

```
=====
```

```
Token List:
```

```
1: (CARGAR, 'cargar')
```

```
2: (CADENA, 'ventas_2024.csv')
```

```
3: (PUNTO_COMA, ';')
```

```
4: (CALCULAR, 'calcular')
```

```
5: (PROMEDIO, 'promedio')
```

```
6: (DE, 'de')
```

```
7: (ID, 'precio')
```

```
8: (PUNTO_COMA, ';')
```

```
...
```

```
64: (MOSTRAR, 'mostrar')
```

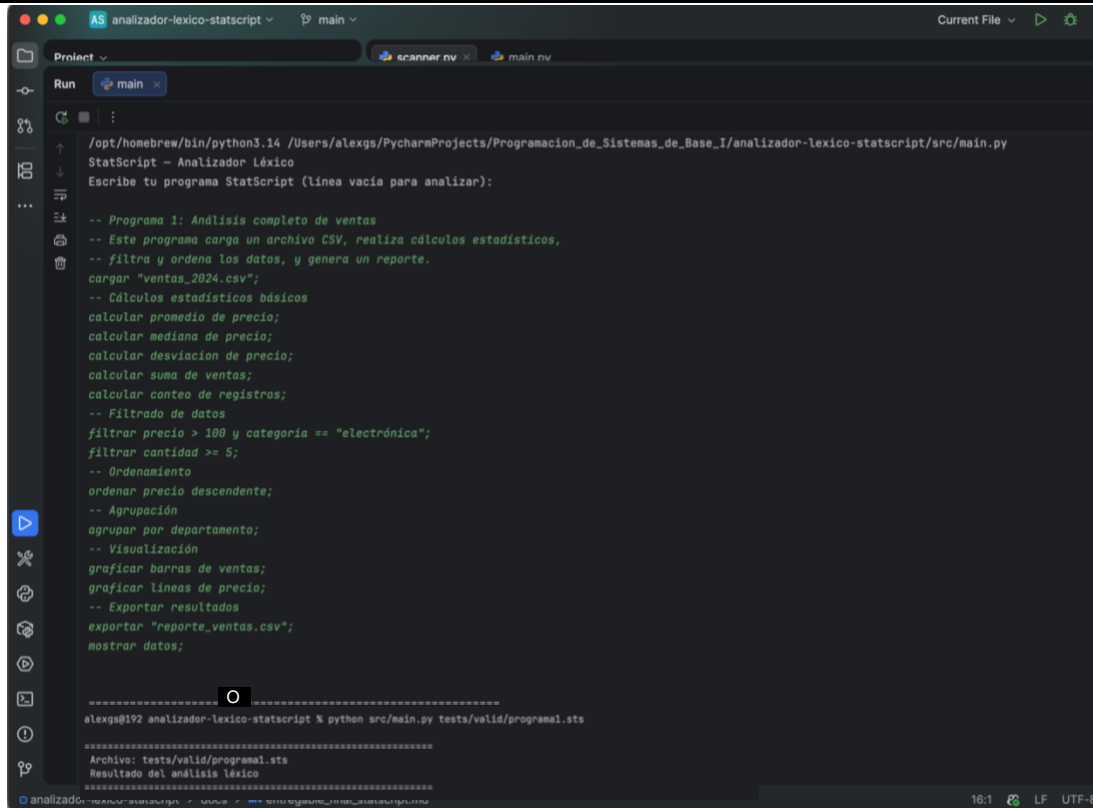
```
65: (DATOS, 'datos')
```

```
66: (PUNTO_COMA, ';')
```

```
-----
```

Error List:

Sin errores léxicos.



```

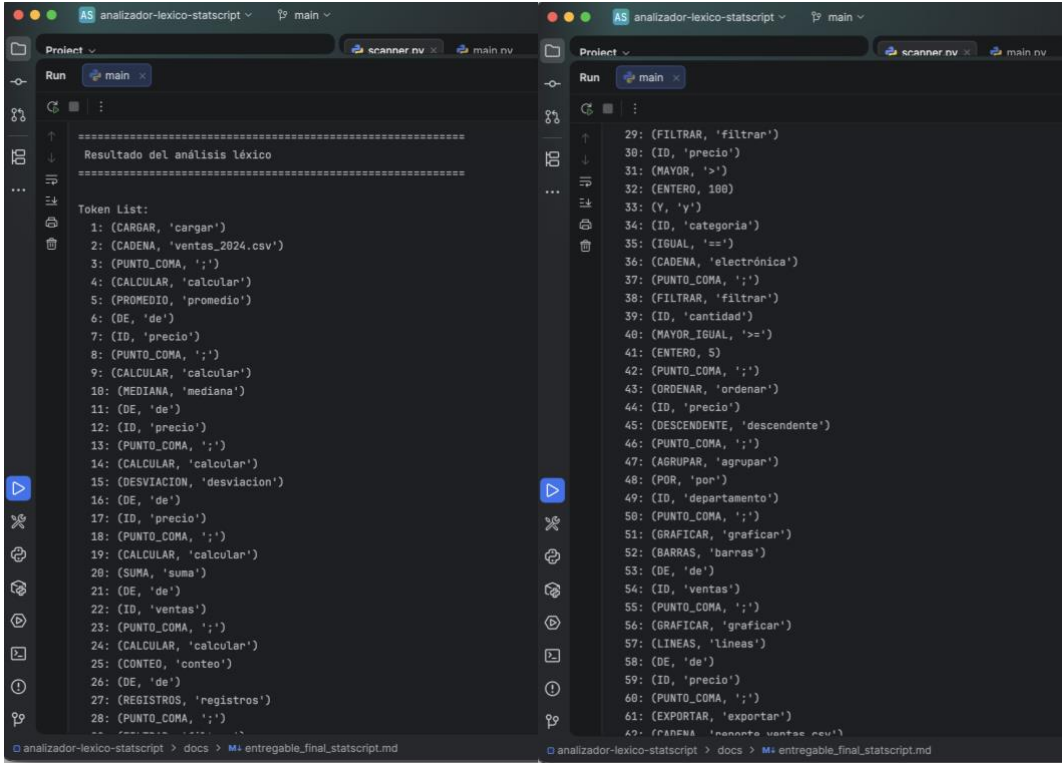
/opt/homebrew/bin/python3.14 /Users/alexgs/PycharmProjects/Programacion_de_Sistemas_de_Base_I/analizador-lexico-statscript/src/main.py
StatScript - Analizador Léxico
Escribe tu programa StatScript (línea vacía para analizar):

-- Programa 1: Análisis completo de ventas
-- Este programa carga un archivo CSV, realiza cálculos estadísticos,
-- filtra y ordena los datos, y genera un reporte.
cargar "ventas_2024.csv";
-- Cálculos estadísticos básicos
calcular promedio de precio;
calcular mediana de precio;
calcular desviación de precio;
calcular suma de ventas;
calcular conteo de registros;
-- Filtrado de datos
filtrar precio > 100 y categoría == "electrónica";
filtrar cantidad >= 5;
-- Ordenamiento
ordenar precio descendente;
-- Agrupación
agrupar por departamento;
-- Visualización
graficar barras de ventas;
graficar líneas de precio;
-- Exportar resultados
exportar "reporte_ventas.csv";
mostrar datos;

=====
alexgs@192 analizador-lexico-statscript % python src/main.py tests/valid/programa1.sts
=====
Archivo: tests/valid/programa1.sts
Resultado del análisis léxico
=====

```

Captura 5.1a — capturas/programa_valido_tokens/captura01a.png



Left Screenshot (Captura 5.1b):

```

=====
Resultado del análisis léxico
=====

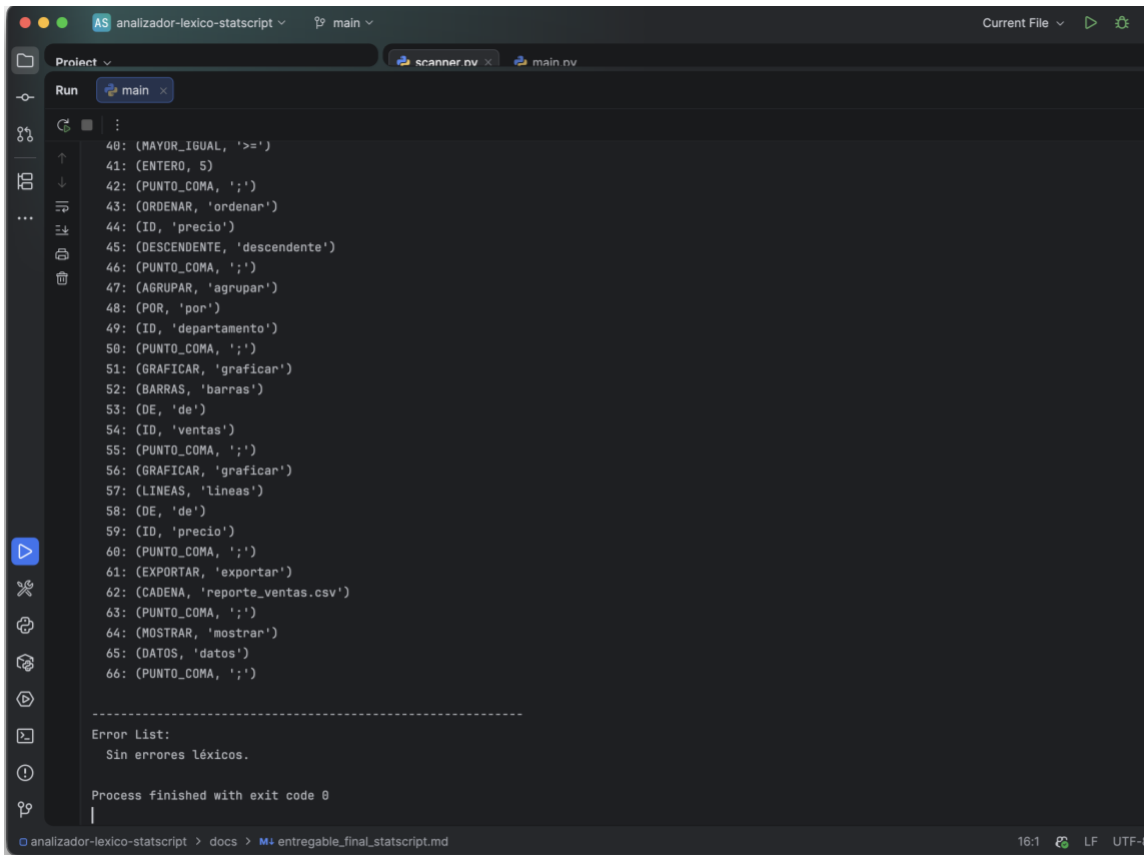
Token List:
1: (CARGAR, 'cargar')
2: (CADENA, 'ventas_2024.csv')
3: (PUNTO_COMA, ',')
4: (CALCULAR, 'calcular')
5: (PROMEDIO, 'promedio')
6: (DE, 'de')
7: (ID, 'precio')
8: (PUNTO_COMA, ',')
9: (CALCULAR, 'calcular')
10: (MEDIANA, 'mediana')
11: (DE, 'de')
12: (ID, 'precio')
13: (PUNTO_COMA, ',')
14: (CALCULAR, 'calcular')
15: (DESVIACION, 'desviacion')
16: (DE, 'de')
17: (ID, 'precio')
18: (PUNTO_COMA, ',')
19: (CALCULAR, 'calcular')
20: (SUMA, 'suma')
21: (DE, 'de')
22: (ID, 'ventas')
23: (PUNTO_COMA, ',')
24: (CALCULAR, 'calcular')
25: (CONTEO, 'conteo')
26: (DE, 'de')
27: (REGISTROS, 'registros')
28: (PUNTO_COMA, ',')
  
```

Right Screenshot (Captura 5.1c):

```

29: (FILTRAR, 'filtrar')
30: (ID, 'precio')
31: (MAYOR, '>')
32: (ENTERO, 100)
33: (Y, 'y')
34: (ID, 'categoria')
35: (IGUAL, '==')
36: (CADENA, 'electrónica')
37: (PUNTO_COMA, ',')
38: (FILTRAR, 'filtrar')
39: (ID, 'cantidad')
40: (MAYOR_IGUAL, '>=')
41: (ENTERO, 5)
42: (PUNTO_COMA, ',')
43: (ORDENAR, 'ordenar')
44: (ID, 'precio')
45: (DESCENDENTE, 'descendente')
46: (PUNTO_COMA, ',')
47: (AGRUPAR, 'agrupar')
48: (POR, 'por')
49: (ID, 'departamento')
50: (PUNTO_COMA, ',')
51: (GRAFICAR, 'graficar')
52: (BARRAS, 'barras')
53: (DE, 'de')
54: (ID, 'ventas')
55: (PUNTO_COMA, ',')
56: (GRAFICAR, 'graficar')
57: (LINEAS, 'lineas')
58: (DE, 'de')
59: (ID, 'precio')
60: (PUNTO_COMA, ',')
61: (EXPORTAR, 'exportar')
62: (CADENA, 'reporte_ventas.csv')
  
```

Captura 5.1b — capturas/programa_valido_tokens/captura01b.png - Captura 5.1c y capturas/programa_valido_tokens/captura01c.png



```

40: (MAYOR_IGUAL, '>=')
41: (ENTERO, 5)
42: (PUNTO_COMA, ',')
43: (ORDENAR, 'ordenar')
44: (ID, 'precio')
45: (DESCENDENTE, 'descendente')
46: (PUNTO_COMA, ',')
47: (AGRUPAR, 'agrupar')
48: (POR, 'por')
49: (ID, 'departamento')
50: (PUNTO_COMA, ',')
51: (GRAFICAR, 'graficar')
52: (BARRAS, 'barras')
53: (DE, 'de')
54: (ID, 'ventas')
55: (PUNTO_COMA, ',')
56: (GRAFICAR, 'graficar')
57: (LINEAS, 'lineas')
58: (DE, 'de')
59: (ID, 'precio')
60: (PUNTO_COMA, ',')
61: (EXPORTAR, 'exportar')
62: (CADENA, 'reporte_ventas.csv')
63: (PUNTO_COMA, ',')
64: (MOSTRAR, 'mostrar')
65: (DATOS, 'datos')
66: (PUNTO_COMA, ',')

-----
Error List:
Sin errores léxicos.

Process finished with exit code 0

```

Captura 5.1d — capturas/programa_valido_tokens/captura01d.png

Como se puede observar en las ilustraciones anteriores, el Scanner clasifica correctamente cada elemento del programa. Las palabras reservadas como CARGAR, CALCULAR y PROMEDIO se reconocen con su tipo correspondiente, los identificadores de usuario se marcan como ID y los comentarios de línea se descartan de forma adecuada, por lo que no aparecen en la lista final de tokens. El mensaje "Sin errores léxicos" al final confirma que el archivo fuente es completamente válido según la gramática de StatScript.

5.2. Ejecución con errores léxicos

En este segundo escenario se tokeniza el archivo `tests/invalid/errores1.sts`, el cual contiene intencionalmente símbolos que no están permitidos por la gramática de StatScript, tales como \$, @, #, ~ y &.

Archivo de entrada — `tests/invalid/errores1.sts`:

```
-- Programa con errores: Símbolos no reconocidos
-- Este archivo contiene múltiples errores léxicos con símbolos que
-- no pertenecen al lenguaje StatScript.

cargar $datos;
filtrar @edad > 30;
calcular #promedio de salario;
mostrar ~resultados;
exportar &reporte;
```

Ejecución y salida:

```
$ python src/main.py tests/invalid/errores1.sts
```

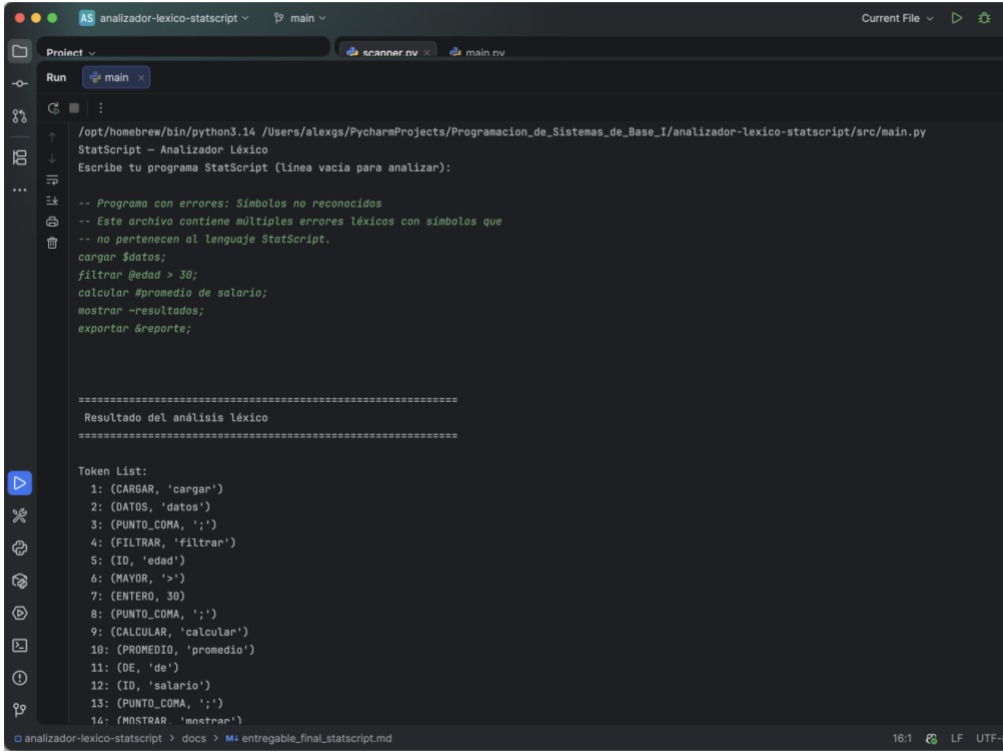
```
=====
Archivo: tests/invalid/errores1.sts
Resultado del análisis léxico
=====
```

Token List:

```
1: (CARGAR, 'cargar')
2: (DATOS, 'datos')
3: (PUNTO_COMA, ';')
4: (FILTRAR, 'filtrar')
5: (ID, 'edad')
6: (MAYOR, '>')
7: (ENTERO, 30)
8: (PUNTO_COMA, ';')
...
19: (PUNTO_COMA, ';')
```

Error List:

Línea 5, Columna 8: Símbolo no reconocido '\$'
 Línea 6, Columna 9: Símbolo no reconocido '@'
 Línea 7, Columna 10: Símbolo no reconocido '#'
 Línea 8, Columna 9: Símbolo no reconocido '~'
 Línea 9, Columna 10: Símbolo no reconocido '&'



```

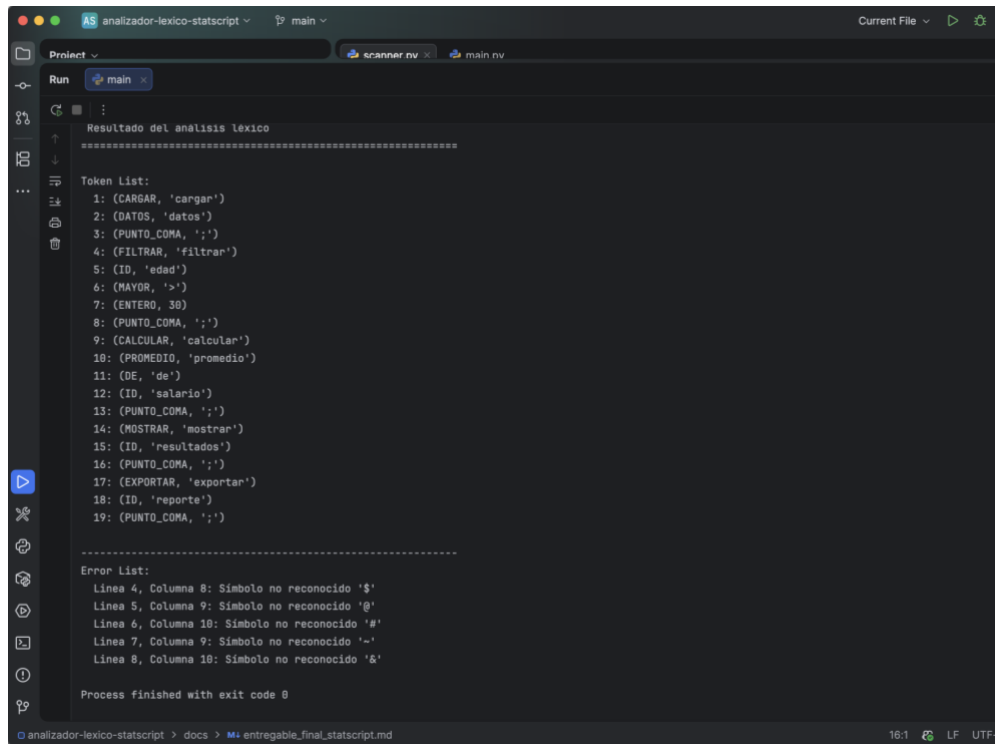
/opt/homebrew/bin/python3.14 /Users/alexgs/PycharmProjects/Programacion_de_Sistemas_de_Base_I/analizador-lexico-statscript/src/main.py
StatScript - Analizador Léxico
Escribe tu programa StatScript (línea vacía para analizar):

-- Programa con errores: Símbolos no reconocidos
-- Este archivo contiene múltiples errores léxicos con símbolos que
-- no pertenecen al lenguaje StatScript.
cargar $datos;
filtrar @edad > 30;
calcular #promedio de salario;
mostrar ~resultados;
exportar &reporte;

=====
Resultado del análisis léxico
=====

Token List:
1: (CARGAR, 'cargar')
2: (DATOS, 'datos')
3: (PUNTO_COMA, ';')
4: (FILTRAR, 'filtrar')
5: (ID, 'edad')
6: (MAYOR, '>')
7: (ENTERO, 30)
8: (PUNTO_COMA, ';')
9: (CALCULAR, 'calcular')
10: (PROMEDIO, 'promedio')
11: (DE, 'de')
12: (ID, 'salario')
13: (PUNTO_COMA, ';')
14: (MOSTRAR, 'mostrar')
  
```

Captura 5.2a — capturas/errores_simbolos/captura02a.png



```

Resultado del análisis léxico
=====
Token List:
1: (CARGAR, 'cargar')
2: (DATOS, 'datos')
3: (PUNTO_COMA, ';')
4: (FILTRAR, 'filtrar')
5: (ID, 'edad')
6: (MAYOR, '>')
7: (ENTERO, 30)
8: (PUNTO_COMA, ';')
9: (CALCULAR, 'calcular')
10: (PROMEDIO, 'promedio')
11: (DE, 'de')
12: (ID, 'salario')
13: (PUNTO_COMA, ';')
14: (MOSTRAR, 'mostrar')
15: (ID, 'resultados')
16: (PUNTO_COMA, ';')
17: (EXPORTAR, 'exportar')
18: (ID, 'reporte')
19: (PUNTO_COMA, ';')

=====
Error List:
Linea 4, Columna 8: Símbolo no reconocido '$'
Linea 5, Columna 9: Símbolo no reconocido '@'
Linea 6, Columna 10: Símbolo no reconocido '#'
Linea 7, Columna 9: Símbolo no reconocido '-'
Linea 8, Columna 10: Símbolo no reconocido '&'

Process finished with exit code 0

```

Captura 5.2b — capturas/errores_simbolos/captura02b.png

Las ilustraciones de este escenario muestran que el Scanner detecta los cinco símbolos inválidos y reporta su ubicación exacta en términos de línea y columna. Resulta importante destacar que el análisis no se detiene ante el primer error, sino que continúa procesando el resto del archivo hasta completar la tokenización. Gracias a este comportamiento, los tokens válidos como CARGAR, DATOS y FILTRAR se reconocen correctamente a pesar de la presencia de los errores, lo cual permite al usuario identificar y corregir todos los problemas de su código fuente en una sola ejecución.

5.3. Caso con identificadores inválidos y símbolos desconocidos

Este tercer escenario representa la prueba más exhaustiva, ya que el archivo `tests/invalid/errores2.sts` combina de manera intencional los cuatro tipos de error que el lexer es capaz de detectar. En particular, se incluyen identificadores que comienzan con dígito, una cadena sin cerrar, un símbolo desconocido y un comentario de bloque que nunca se cierra.

Archivo de entrada — `tests/invalid/errores2.sts`:

```
-- Programa con errores: Identificadores inválidos y cadenas sin cerrar

-- Error: identificadores que comienzan con dígitos
calcular promedio de 123abc;
filtrar 456columna > 100;

-- Error: cadena sin cerrar (la comilla nunca se cierra)
cargar "archivo_sin_cerrar.csv;

-- Mezcla de código válido con errores
mostrar datos;
filtrar $precio > 50;
calcular 99total de ventas;
exportar "resultado.csv";

-- Error: comentario de bloque sin cerrar (al final del archivo)
<-- Este comentario nunca se cierra
```

Ejecución y salida:

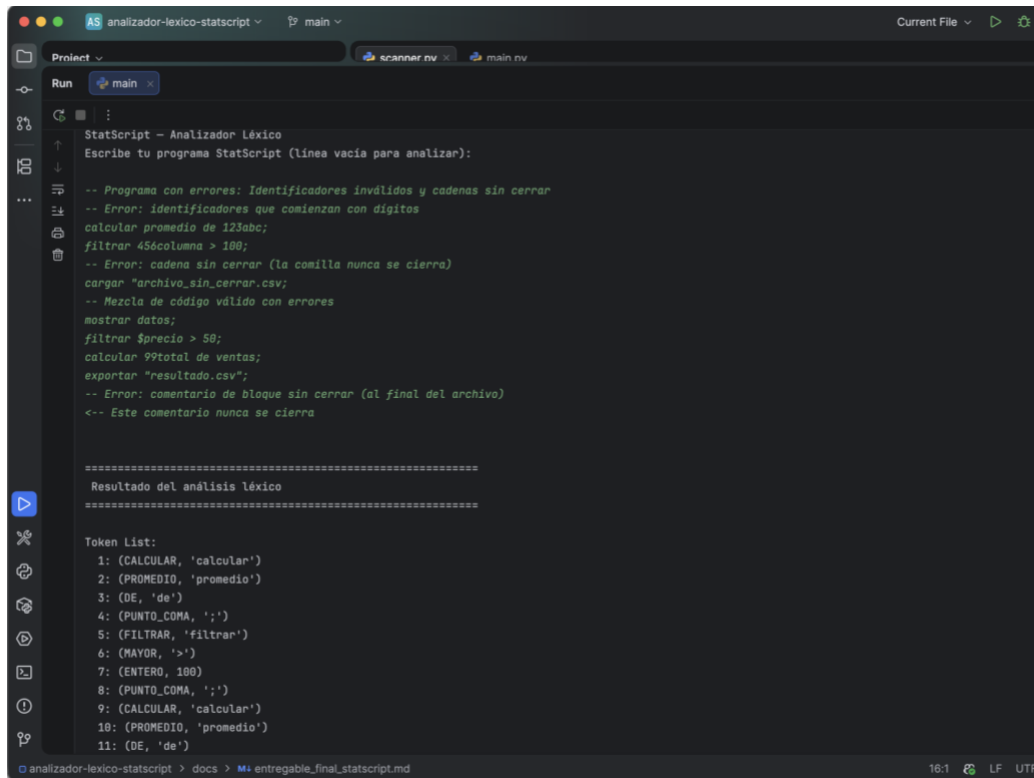
```
$ python src/main.py tests/invalid/errores2.sts

=====
Archivo: tests/invalid/errores2.sts
Resultado del análisis léxico
=====

Token List:
1: (CALCULAR, 'calcular')
2: (PROMEDIO, 'promedio')
3: (DE, 'de')
4: (PUNTO_COMA, ';')
...
23: (CADENA, 'resultado.csv')
24: (PUNTO_COMA, ';')

-----

Error List:
Linea 6, Columna 22: Identificador inválido '123abc'
Linea 7, Columna 9: Identificador inválido '456columna'
Linea 10, Columna 8: Cadena sin cerrar '"archivo_sin_cerrar.csv;'
Linea 14, Columna 9: Símbolo no reconocido '$'
Linea 15, Columna 10: Identificador inválido '99total'
Linea 20, Columna 1: Comentario de bloque sin cerrar '<-- Este comentario...'
```



```

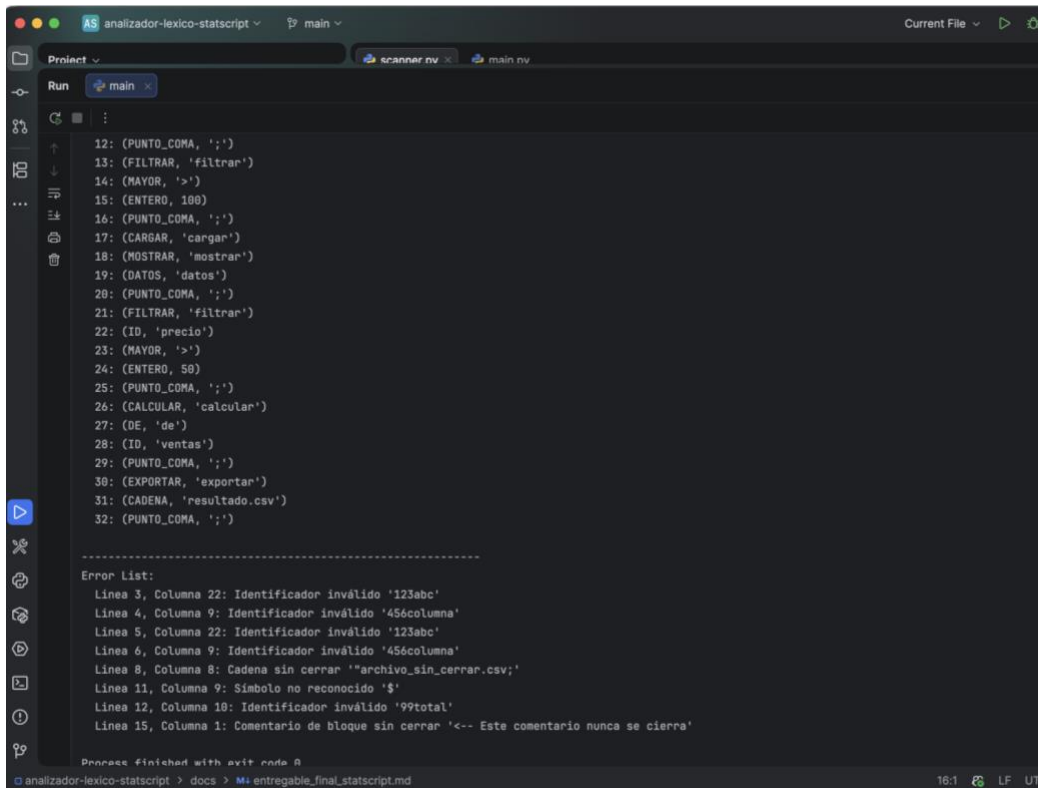
StatScript - Analizador Léxico
Escribe tu programa StatScript (línea vacía para analizar):

-- Programa con errores: Identificadores inválidos y cadenas sin cerrar
-- Error: identificadores que comienzan con dígitos
calcular promedio de 123abc;
filtrar 456columna > 100;
-- Error: cadena sin cerrar (la comilla nunca se cierra)
cargar "archivo_sin_cerrar.csv;
-- Mezcla de código válido con errores
mostrar datos;
filtrar $precio > 50;
calcular 99total de ventas;
exportar "resultado.csv";
-- Error: comentario de bloque sin cerrar (al final del archivo)
<-- Este comentario nunca se cierra

=====
Resultado del análisis léxico
=====

Token List:
1: (CALCULAR, 'calcular')
2: (PROMEDIO, 'promedio')
3: (DE, 'de')
4: (PUNTO_COMA, ',')
5: (FILTRAR, 'filtrar')
6: (MAYOR, '>')
7: (ENTERO, 100)
8: (PUNTO_COMA, ',')
9: (CALCULAR, 'calcular')
10: (PROMEDIO, 'promedio')
11: (DE, 'de')
  
```

Captura 5.3a — capturas/errores_mixtos/captura03a.png



```

12: (PUNTO_COMA, ',')
13: (FILTRAR, 'filtrar')
14: (MAYOR, '>')
15: (ENTERO, 100)
16: (PUNTO_COMA, ',')
17: (CARGAR, 'cargar')
18: (MOSTRAR, 'mostrar')
19: (DATOS, 'datos')
20: (PUNTO_COMA, ',')
21: (FILTRAR, 'filtrar')
22: (ID, 'precio')
23: (MAYOR, '>')
24: (ENTERO, 50)
25: (PUNTO_COMA, ',')
26: (CALCULAR, 'calcular')
27: (DE, 'de')
28: (ID, 'ventas')
29: (PUNTO_COMA, ',')
30: (EXPORTAR, 'exportar')
31: (CADENA, 'resultado.csv')
32: (PUNTO_COMA, ',')

Error List:
Linea 3, Columna 22: Identificador inválido '123abc'
Linea 4, Columna 9: Identificador inválido '456columna'
Linea 5, Columna 22: Identificador inválido '123abc'
Linea 6, Columna 9: Identificador inválido '456columna'
Linea 8, Columna 8: Cadena sin cerrar '"archivo_sin_cerrar.csv;'
Linea 11, Columna 9: Símbolo no reconocido '$'
Linea 12, Columna 10: Identificador inválido '99total'
Linea 15, Columna 1: Comentario de bloque sin cerrar '<-- Este comentario nunca se cierra'

Process finished with exit code 0
  
```

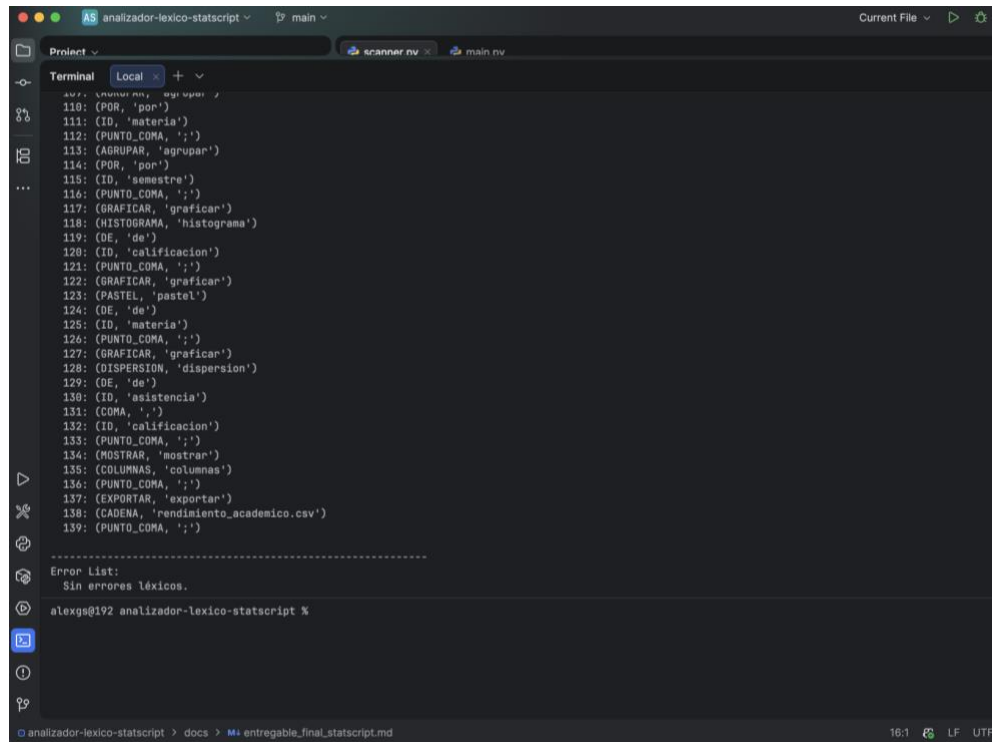
Captura 5.3b — capturas/errores_mixtos/captura03b.png

Este escenario constituye la prueba más completa del sistema de errores, ya que abarca los cuatro tipos contemplados en la especificación. El Scanner identifica con éxito los identificadores que comienzan con dígito, la cadena que no fue cerrada, el símbolo no reconocido y el comentario de bloque sin cierre. En todos los casos, el mensaje de error generado incluye la línea, la columna y una descripción específica del problema, lo que demuestra la robustez del mecanismo de reporte implementado.

5.4. Segundo programa válido

Con el propósito de demostrar que el lexer alcanza una cobertura del 100 % sobre los tipos de token definidos, se ejecuta también el archivo `tests/valid/programa2.sts`. Este programa incorpora comentarios de bloque, las diez funciones estadísticas, los cinco tipos de gráfica, operadores aritméticos, relacionales y lógicos, así como expresiones de asignación, lo cual complementa la cobertura parcial obtenida con `programa1.sts`.

```
$ python src/main.py tests/valid/programa2.sts
```



```

analizador-lexico-statscript % python src/main.py tests/valid/programa2.sts

=====
Archivo: tests/valid/programa2.sts
Resultado del análisis léxico
=====

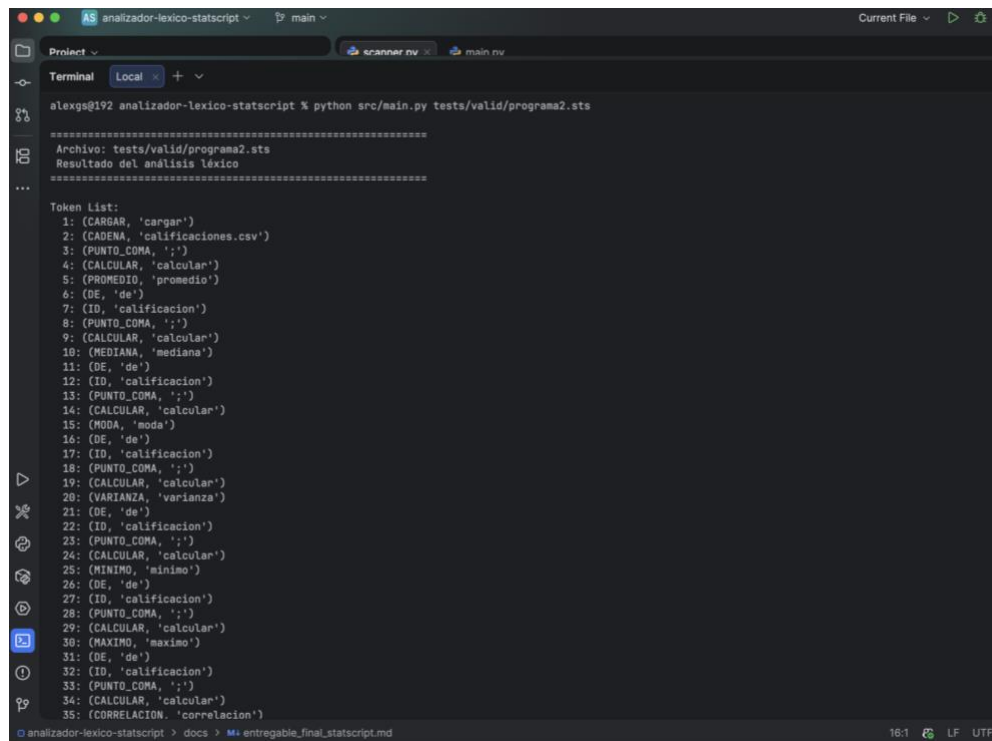
Token List:
1: (CARGAR, 'cargar')
2: (CADENA, 'calificaciones.csv')
3: (PUNTO_COMA, ';')
4: (CALCULAR, 'calcular')
5: (PROMEDIO, 'promedio')
6: (DE, 'de')
7: (ID, 'calificacion')
8: (PUNTO_COMA, ';')
9: (CALCULAR, 'calcular')
10: (MEDIANA, 'mediana')
11: (DE, 'de')
12: (ID, 'calificacion')
13: (PUNTO_COMA, ';')
14: (CALCULAR, 'calcular')
15: (MODA, 'moda')
16: (DE, 'de')
17: (ID, 'calificacion')
18: (PUNTO_COMA, ';')
19: (CALCULAR, 'calcular')
20: (VARIANZA, 'varianza')
21: (DE, 'de')
22: (ID, 'calificacion')
23: (PUNTO_COMA, ';')
24: (CALCULAR, 'calcular')
25: (MINIMO, 'minimo')
26: (DE, 'de')
27: (ID, 'calificacion')
28: (PUNTO_COMA, ';')
29: (CALCULAR, 'calcular')
30: (MAXIMO, 'maximo')
31: (DE, 'de')
32: (ID, 'calificacion')
33: (PUNTO_COMA, ';')
34: (CALCULAR, 'calcular')
35: (CORRELACION, 'correlacion')

=====
Error List:
Sin errores léxicos.

alexgs@192 analizador-lexico-statscript %

```

Captura 5.4a — capturas/programa_valido_2/captura04a.png



```

analizador-lexico-statscript % python src/main.py tests/valid/programa2.sts

=====
Archivo: tests/valid/programa2.sts
Resultado del análisis léxico
=====

Token List:
1: (CARGAR, 'cargar')
2: (CADENA, 'calificaciones.csv')
3: (PUNTO_COMA, ';')
4: (CALCULAR, 'calcular')
5: (PROMEDIO, 'promedio')
6: (DE, 'de')
7: (ID, 'calificacion')
8: (PUNTO_COMA, ';')
9: (CALCULAR, 'calcular')
10: (MEDIANA, 'mediana')
11: (DE, 'de')
12: (ID, 'calificacion')
13: (PUNTO_COMA, ';')
14: (CALCULAR, 'calcular')
15: (MODA, 'moda')
16: (DE, 'de')
17: (ID, 'calificacion')
18: (PUNTO_COMA, ';')
19: (CALCULAR, 'calcular')
20: (VARIANZA, 'varianza')
21: (DE, 'de')
22: (ID, 'calificacion')
23: (PUNTO_COMA, ';')
24: (CALCULAR, 'calcular')
25: (MINIMO, 'minimo')
26: (DE, 'de')
27: (ID, 'calificacion')
28: (PUNTO_COMA, ';')
29: (CALCULAR, 'calcular')
30: (MAXIMO, 'maximo')
31: (DE, 'de')
32: (ID, 'calificacion')
33: (PUNTO_COMA, ';')
34: (CALCULAR, 'calcular')
35: (CORRELACION, 'correlacion')

=====
Error List:
Sin errores léxicos.

alexgs@192 analizador-lexico-statscript %

```

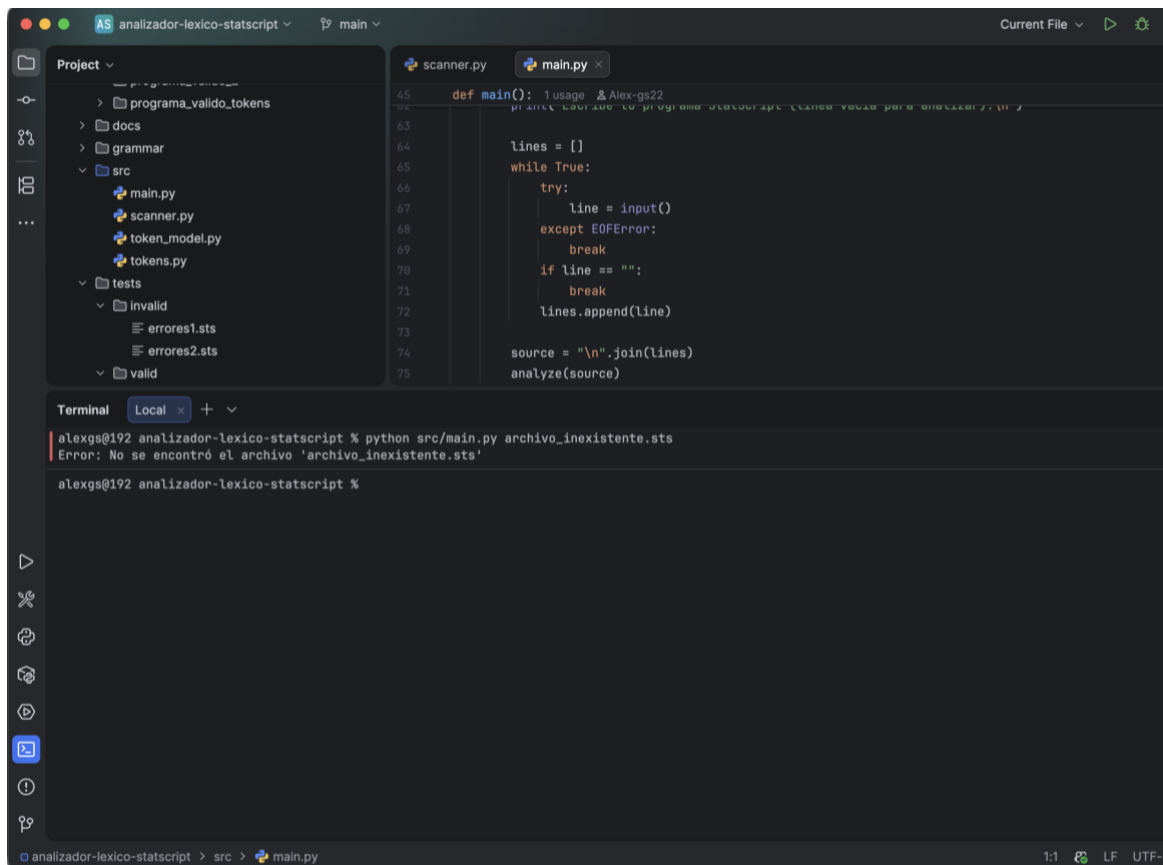
Captura 5.4d — capturas/programa_valido_2/captura04d.png

La ejecución de este segundo programa válido complementa las pruebas del escenario 5.1, ya que entre ambos archivos se cubren la totalidad de los 33 tipos de palabra reservada y todos los operadores definidos en la gramática de StatScript. Esta cobertura completa garantiza que el lexer no presenta omisiones en el reconocimiento de tokens.

5.5. Manejo de archivo no encontrado

Para verificar que el programa maneja correctamente los errores de entrada/salida, se invoca el lexer con un archivo que no existe en el sistema de archivos.

```
$ 34python src/main.py archivo_inexistente.sts
Error: No se encontró el archivo 'archivo_inexistente.sts'
```



```
def main():
    """
    1 usage: & Alex-gs22
    """
    print("Ejecutando el programa-lexico-sts. Cargando vocabulario de palabras reservadas.")

    lines = []
    while True:
        try:
            line = input()
        except EOFError:
            break
        if line == "":
            break
        lines.append(line)

    source = "\n".join(lines)
    analyze(source)
```

```
alexgs@192 analizador-lexico-statscript % python src/main.py archivo_inexistente.sts
Error: No se encontró el archivo 'archivo_inexistente.sts'

alexgs@192 analizador-lexico-statscript %
```

Captura 5.5a — capturas/archivo_inexistente/captura05a.png

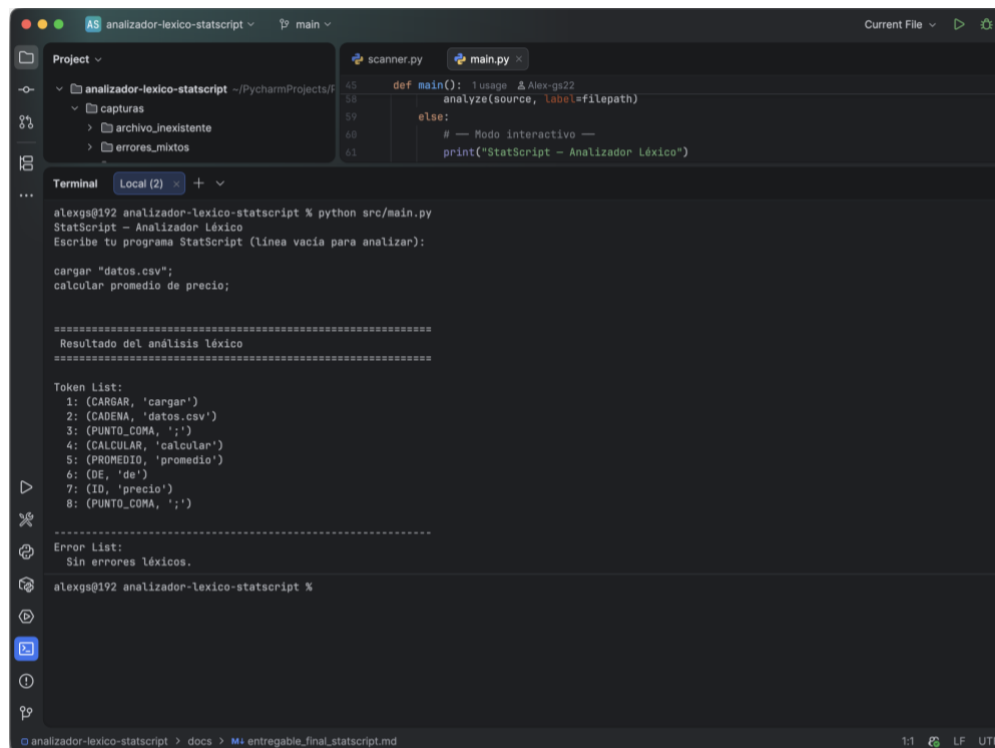
Como se observa en la captura, el programa detecta que el archivo solicitado no existe y responde con un mensaje de error descriptivo y limpio, sin generar el traceback técnico de Python. Este comportamiento evidencia un manejo robusto de excepciones que mejora la experiencia del usuario al utilizar la herramienta desde la terminal.

5.6. Modo consola

El lexer también puede ejecutarse sin argumentos tal y como se muestra en el punto 5.1 activando un modo interactivo en el que el usuario escribe instrucciones directamente en la terminal y obtiene el resultado al presionar Enter en una línea vacía.

```
$ python src/main.py
StatScript — Analizador Léxico
Escribe tu programa StatScript (línea vacía para analizar):
```

```
cargar "datos.csv";
calcular promedio de precio;
```



```

def main():
    if __name__ == '__main__':
        analyze(source, label='filepath')
    else:
        # --- Modo interactivo ---
        print("StatScript — Analizador Léxico")

Token List:
1: (CARGAR, 'cargar')
2: (CADENA, 'datos.csv')
3: (PUNTO_COMA, ';')
4: (CALCULAR, 'calcular')
5: (PROMEDIO, 'promedio')
6: (DE, 'de')
7: (ID, 'precio')
8: (PUNTO_COMA, ';')

Error List:
Sin errores léxicos.

```

Captura 5.6a — capturas/modo_consola/captura06a.png

En esta última prueba se demuestra que el lexer no depende exclusivamente de archivos para funcionar. El usuario puede escribir instrucciones de StatScript directamente en la terminal y obtener el análisis léxico al presionar Enter en una línea vacía, lo que resulta útil para pruebas rápidas durante el desarrollo o para demostrar el funcionamiento del sistema sin necesidad de crear archivos adicionales.

6. Conclusiones

Cabrales Cruz Raciél Julian. Este proyecto me permitió entender de primera mano cómo funciona la fase inicial de un compilador, algo que antes solo conocía de manera teórica. La parte que más me costó fue comprender cómo el Scanner decide entre un operador simple y uno compuesto, ya que la lógica de lookahead no es tan intuitiva como parece al principio. Sin embargo, una vez que entendí el flujo de `peek()` y `match()`, todo cobró sentido. Si pudiera mejorar algo, me gustaría haber implementado mensajes de error aún más descriptivos, quizá sugiriendo al usuario qué quiso escribir.

Guzmán Sánchez Alex Arath. Lo que más valoro de este proyecto es haber construido el lexer sin depender de herramientas generadoras como ANTLR o Flex. Hacerlo de forma manual me obligó a pensar en cada caso particular, desde los comentarios de bloque multilínea hasta la detección de identificadores que empiezan con dígitos. El reto más grande fue lograr que el Scanner siguiera funcionando después de encontrar un error, en lugar de detenerse. Aprendí que la modularidad del código no es un lujo sino una necesidad cuando el proyecto crece, y eso es algo que planeo aplicar en futuros desarrollos.

Muñoz Perales Luis Gonzalo. Antes de este proyecto, no tenía una idea clara de lo que implica diseñar un lenguaje desde cero, por pequeño que sea. Definir las reglas léxicas de StatScript y traducirlas a expresiones regulares me ayudó a consolidar conceptos que había visto en clase pero nunca había aplicado. La parte más complicada para mí fue el manejo de las cadenas sin cerrar, porque requiere decidir en qué punto exacto se reporta el error. Creo que el resultado final es sólido y me deja con una base práctica para abordar la construcción de un parser en el futuro.

Saenz Rico Perez Marco Fernando. Mi principal aprendizaje fue entender la importancia de las pruebas en el desarrollo de software. Diseñar los archivos de prueba con errores intencionales me hizo pensar en escenarios que normalmente no consideraría, como qué pasa cuando un comentario de bloque nunca se cierra o cuando alguien escribe 123abc como nombre de variable. También me di cuenta de que un buen mensaje de error puede marcar la diferencia entre una herramienta útil y una frustrante. Si tuviera más tiempo, exploraría la posibilidad de agregar un modo de corrección automática que sugiera alternativas al usuario.

Trabajo futuro

De cara a futuras iteraciones del proyecto, se identifican varias líneas de mejora que podrían enriquecer tanto la funcionalidad como la experiencia de uso de StatScript.

1. Implementar un analizador sintáctico (parser) que consuma la lista de tokens generada por el lexer y construya un árbol de sintaxis abstracta (AST), completando así la segunda fase del compilador.
2. Agregar soporte para cadenas multilínea mediante el uso de comillas triples ("""), lo que facilitaría la definición de consultas largas.
3. Incorporar números en notación científica ($1.5e10$) para ampliar las capacidades numéricas del lenguaje.
4. Desarrollar un intérprete que ejecute los programas StatScript directamente sobre archivos CSV reales, transformando el lenguaje de un ejercicio académico a una herramienta funcional.

7. Referencias

- [1] Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2.a ed.). Pearson Addison-Wesley.
- [2] Nystrom, R. (2021). *Crafting Interpreters*. Genever Benning. Recuperado de <https://craftinginterpreters.com/>
- [3] Python Software Foundation. (2024). *re — Regular expression operations*. Recuperado de <https://docs.python.org/3/library/re.html>
- [4] Python Software Foundation. (2024). *dataclasses — Data Classes*. Recuperado de <https://docs.python.org/3/library/dataclasses.html>
- [5] Python Software Foundation. (2024). *sys — System-specific parameters and functions*. Recuperado de <https://docs.python.org/3/library/sys.html>
- [6] Real Python. (2024). *How to Write a Lexer in Python*. Recuperado de <https://realpython.com/>
- [7] GeeksforGeeks. (2024). *Introduction to Lexical Analysis*. Recuperado de <https://www.geeksforgeeks.org/introduction-of-lexical-analysis/>